

wx-cli-again 小白指南

微信本地数据命令行工具 (Rust) —— 从 DMCA 下架到重生的开源故事

拾遗学习 · task 1c5dba8b · 2026-10-03

Rust

SQLCipher

daemon 缓存

.dat 三档解码

AI Skill

目录 (疑问即卡片)

Q1 这条推文背后发生了什么？wx-cli 的 DMCA 下架与重生

Q2 wx-cli-again 是什么？解决什么问题？

Q3 为什么查询能毫秒级？——daemon + mtime 缓存架构

Q4 SQLCipher 「在线打开」和「全量解密」差在哪？

Q5 微信的 message_N.db 分片是什么结构？

Q6 聊天图片 .dat 为什么有三档解码？

Q7 legacy XOR 连 magic 都没有，怎么知道密钥？

Q8 输出里包的 meta wrapper 是给谁看的？

Q9 「不需要关 SIP」是真的吗？

Q10 五方位对比：它和已学的微信赛道项目差在哪？

Q11 隐私与法律红线在哪里？

Q12 现在能装吗？发布渠道现状

逐条解答

Q1 这条推文背后发生了什么？

推文作者 kabikabi (GitHub: jackwener) 说"太多人找我要 wx-cli 了，再度 public 一下"——背景是：原版 `jackwener/wx-cli` 在 2026-07-15 被 GitHub 以 DMCA 理由封锁（投诉方指向微信，gh API 实测返回 451 `Repository access blocked (reason: dmca)`）。作者随后把代码以 `wx-cli-again` 名义重新公开（Apache-2.0, 771 stars），并声明灵感来自 ylytdeng/wechat-decrypt。



诚实记录：本指南学习对象是公开的 `jackwener/wx-cli-again`；原仓库的完整历史已不可访问，两者差异只能从 README 自述（"fresh start from wx-cli"）推断。

Q2 wx-cli-again 是什么？解决什么问题？

一句话：一个 Rust 写的单一二进制命令行工具，查询/解密/导出你自己电脑上的微信 4.x 本地数据——会话、聊天记录、全文搜索、联系人、群成员、朋友圈、公众号文章、收藏、统计、附件图片导出，全部输出 YAML/JSON，还专门给 AI Agent 设计了一套 `SKILL.md`（`npx skills add` 一键装进 Claude Code / Cursor / Codex）。

没有它之前：想翻自己的微信聊天记录只能靠微信自带搜索，弱、慢、无法批量导出。它把"个人数据自己拿回来"做成了类 Unix 的命令行体验：`wx history "张三"`、`wx search "关键词"`、`wx export "AI群" --format markdown`。

类比已学项目：它和 `[[wechat-ai-memory]]`（导出 PDF/JSON 喂给 Agent）、`[[wechat-intelligence-hub]]`（情报信号引擎）是同一层的工具——但 wx-cli 更"基建"：不预设记忆/情报用途，只做通用数据访问层 + 毫秒级查询服务。

Q3 为什么查询能毫秒级？——daemon 缓存架构

核心设计：**CLI 本身不干活，只是客户端**。首次查询时自动拉起后台 `wx-daemon` 进程，daemon 解密数据库后把明文库 + mtime 索引持久化到 `~/.wx-cli/cache/`。之后每次查询通过 Unix socket (Windows 是 named pipe) 发请求，**mtime 没变就直接复用缓存**，毫秒级返回。



对比：wechat-ai-memory 每次导出都要重新解密一遍；wx-cli 把"解密"摊销到 daemon 的生命周期里——查 100 次只解密 1 次。

Q4 SQLCipher「在线打开」和「全量解密」差在哪？

微信 4.x 的每个 .db 都是 **SQLCipher 4 整库加密** (AES-256-CBC + HMAC-SHA512)。传统做法（如 [[进程取钥与副本解密]] 描述的 Frida 路线）：拿到 32 字节密钥 → 把整个库解密复制一份 → 在副本上查询。wx-cli 的做法更激进：**用 rusqlite 的 bundled SQLCipher 直接对加密库"在线打开"**——PRAGMA key 以 raw-key 语法 `x'<64hex>'` 喂进去（跳过 PBKDF2），SQLite 每读一页就地解密，**不落任何明文副本**；只有在线打不开时才降级为解密到 cache。

源码依据：`src/crypto/sqlcipher.rs` 的 `open_encrypted_readonly()`——只读标志 + `PRAGMA query_only = ON`，先尝试 key-only，失败再回退 key+salt 拼接。

类比：全量解密像"把整本外文原著翻译后另存一个文件"；在线打开像"带着词典读原版"——页级 (page) 随读随译。

Q5 message_N.db 分片是什么结构?

微信把聊天记录按时间和体量切成多个分片库: `message/message_1.db`、

`message_2.db` ...还有独立的 `biz_message_*.db` (公众号文章) 和 `message_resource.db`

(附件索引)。每个分片有独立的 32 字节密钥——旧消息所在的分片可能长期不被微信加载, 密钥也就迟迟不出现在内存里, 这就是"冷分片缺钥"问题。wx-cli 的对策是

`meta.unknown_shards`: 查询结果里明确告诉你哪些分片没密钥、结果可能不全

(`possibly_stale` 状态), 而不是悄悄返回残缺的"假完整"。

message_1.db

热分片·有钥 ✓

message_2.db

热分片·有钥 ✓

message_3.db

冷分片·缺钥 ⚠

biz_msg

公众号

resource

附件索引

`meta.unknown_shards = ["message_3.db"]` → 结果标记 `possibly_stale`, 不装完整

Q6 聊天图片 .dat 为什么有三档解码？

微信存的聊天图片不是 jpg/png 原文件，而是 `msg/attach/.../*.dat` 加密文件，而且历史上换过三次加密方案，工具必须兼容全部：

档位	文件头 magic	算法	密钥来源
legacy XOR	无（靠内容反推）	全文件单字节 XOR	用已知图片 magic 反推
V1 固定 AES	<code>07 08</code> <code>V1 08</code> <code>07</code>	AES-128-ECB (PKCS7)	硬编码常量 <code>cfdc208495d565ef</code> (= md5("0") 前 16 位)
V2 AES + XOR	<code>07 08</code> <code>V2 08</code> <code>07</code>	三段拼接：AES-ECB + 明文 raw + 单字节 XOR	平台派生 image AES key (macOS 从 kvcomm cache / 暴力枚举 uin; Windows 扫 WeChat.exe 内存)



Q7 legacy XOR 没有 magic，怎么知道密钥？

妙招：用“正确答案”反推。图片格式都有固定文件头（PNG 以 `89 50 4E 47` 开头、JPG 以 `FF D8 FF` 开头...），假设明文第一个字节是 `magic[0]`，那么 `XOR key = 密文[0] ⊕ magic[0]`；拿这个 key 去验 magic 剩余字节，全部命中才算数。按 magic 长度降序尝试（PNG 4 字节优先于 JPG 3 字节），避免短 magic 的假阳性；BMP 只有 2 字节 magic，还要额外校验文件头里的 `bf_size`/`bf_offset` 合理性。

本批次实验 1 实测验证：合成 PNG 用任意 key `0x5A` XOR 成伪 .dat，上游真实源码 `dispatch()` 自动反推出 key 并无损还原 87 字节原图（见动手实验）。

Q8 meta wrapper 是给谁看的?

`history` / `search` / `sessions` 等命令的 stdout 不是裸数组, 而是 `{data, meta}` 包装: `meta.status` (ok / possibly_stale...)、`meta.unknown_shards`、`chat_latest_timestamp` vs `session_last_timestamp` (对账: session.db 里微信自己记的最新时间若领先于查到的, 说明漏消息了)。这是写给 AI Agent 看的"数据新鲜度声明"——agent 消费输出时自己判断能不能信、缺了哪块, 不用人类盯 stderr。配套还有 SKILL.md / AGENTS.md, 让这个 CLI 可以被 Claude Code 当技能安装。

Q9 「不需要关 SIP」是真的吗?

按 README 与 `src/scanner/macos.rs` 注释: 取钥两阶段 (① `task_for_pid` 进程内存扫描搜 `x'key+salt'` 模式; ② LLDB hook `CCCryptorCreate` 补冷分片密钥) 都不依赖关闭 SIP——SIP 保护的是系统组件, 不是"能不能读微信内存"的开关。需要的是本机 GUI Terminal + sudo (Hardened Runtime 官方包) + 在"隐私与安全性 → 开发者工具"里授权终端。

诚实记录: 本机是 Windows 11, 没有 macOS/SIP/LLDB 环境, 该声明无法在本机复验, 只转述源码注释与 README。Windows 版取钥是扫 WeChat.exe 内存匹配 V2 key 候选, 同样未在本机复验 (无微信 4.x 测试环境)。

Q10 五方位对比：微信赛道项目

	wx-cli-again	yichen-skills wechat- local-vault	wechat-ai- memory	wechat- intelligence- hub	Memmy
定位	通用数据访问层 CLI + AI Skill	Skill 合集工 作流	记忆流水线进料口 (导出三件套)	情报库 (信号引 擎)	记忆仓库
语言/ 形态	Rust 单二进制 + daemon	Python 脚本 集 + Frida	Python/PySide6 GUI	(见对应笔记)	(见对应 笔记)
取 数 路 线	task_for_pid 内存 扫描 + LLDB hook (不关 SIP)	Frida 取钥 ([[进程取钥 与副本解 密]])	Frida 取钥	授权 salt-key ([[CleanRoom 只读读取器]])	—
独 门	daemon mtime 缓存毫秒级 + 在线 打开 SQLCipher 不落明文 + .dat 三档解码 + meta 新鲜度声明	工程化护栏最 全 (授权硬 门/快照契 约)	适配器隔离 + JSON 备用入口	边界最干净 (不碰 进程)	记忆组织 分层
平 台	macOS / Linux / Windows 三平台	Mac 主场 + Windows 快 照实验性	Windows	—	—
护 栏 风 格	只读 + query_only + 免 责声明 (口头契 约)	写进代码契约 (WAL 拒 收/UUIDv4 manifest)	确定性护栏	授权材料制	—

一句话：wx-cli-again 是赛道里“工程性能最强、平台最广”的选手；护栏精细度不如 yichen-skills，边界干净度不如 intelligence-hub，但它把查询体验做成了基础设施级。

Q11 隐私与法律红线在哪？

- README 免责声明：仅限解密**自己的**微信数据，用于学习研究；原仓库被 DMCA 下架就是越过红线的代价写照。
- 技术层护栏：数据库只读打开 + `PRAGMA query_only = ON`；密钥存本机 `~/wx-cli/all_keys.json`，不联网。
- 本批次学习全程**零真实微信数据**：实验只用 Pillow 合成的 87 字节 PNG 走编解码往返（对比 wechat-ai-memory / yichen-skills 两批次的同一红线做法）。

Q12 现在能装吗？

现实有点绕：README 宣称的"源码与发布仓库" `botiverse/wx-cli` 当前是 **private**，匿名 curl/旧 npm 包 `@jackwener/wx-cli@0.3.0` 拿不到新二进制。**可行路径只有一条：源码自编译**——而本批次已在 Windows 11 实测走通（见下节实验，含 MinGW 坑）。Windows 上微信 4.x 取钥支持是"扫 WeChat.exe 内存匹配 V2 候选"，比 macOS 路线年轻，稳定性预期低一些。

这个项目给我什么能力

- **秒查自己的聊天历史**：`wx history "张三" -n 2000`、按时间段过滤、全文 `wx search`——一旦配好密钥，比微信自带搜索强一个量级。
- **给 Agent 接上微信数据源**：meta wrapper + SKILL.md 是专门为 LLM 消费设计的，装上技能后可以让 Claude Code 直接查你的聊天上下文。
- **导出归档**：markdown / JSON 导出任意会话，时间范围可控，适合做项目复盘材料。
- **朋友圈/公众号/收藏检索**：三个常被忽视的本地数据源（sns-feed、sns-search、biz-articles、favorites）。
- **.dat 图片抢救**：三档解码器兼容微信历代图片加密，能把自己的聊天图片批量还原成 jpg/png。
- **学习样本**：即使不用微信，它的"daemon 缓存 + 在线解密 + AI 友好输出"架构也是本地数据 CLI 的范本（本批次 decoder 实验可直接搬运其源码）。

动手实验：编译 wx-cli + 用上游真实解码器源码做合成数据往返（全程零真实微信数据）

步骤 1 • 装 Rust 工具链（约 3 分钟）

操作： `curl -sSf https://sh.rustup.rs | sh -s -- -y --profile minimal` 目的：拿到 cargo/rustc 验证： `cargo --version` 有版本输出
✓ 预期结果： `cargo 1.xx.x`

步骤 2 • 克隆并编译 wx-cli-again（Windows 首个坑在此）

操作： `git clone https://github.com/jackwener/wx-cli-again && cd wx-cli-again && cargo build --release` 目的：得到单一二进制 `wx.exe`

本机实测坑：rustup 默认装的是 GNU 工具链，链接时调用

`D:\MinGW\bin\ld.exe` 报 `Invalid bfd target, getrandom / windows-sys` 编译失败。解法：本机已有 VS BuildTools 18，改用 MSVC 工具链——`rustup toolchain install stable-x86_64-pc-windows-msvc`，然后在 `vcvars64.bat` 环境里 `rustup default stable-x86_64-pc-windows-msvc && cargo build --release` 即通过。

✓ 预期结果： `target\release\wx.exe` 生成（约 6MB 单文件）

步骤 3 • 跑真实二进制（无微信环境 = 诚实输出）

操作： `wx --version`、`wx --help`、`wx doctor` 目的：验证二进制可用、看健康检查在"零密钥"环境下的真实表现

```
F:\reminder\wx_cli_again\repo\target\release> wx.exe --version  
wx 0.6.3
```

```
F:\reminder\wx_cli_again\repo\target\release> wx.exe doctor  
X WeChat 进程                    未运行  
X config.json                   未找到，请先 wx init  
X daemon socket                未运行（首次查询会自动启动）  
  
存在未通过项，补密钥：sudo wx key extract --hook-seconds 90  
（本机 GUI Terminal + 等待时打开相关聊天；SIP 无需关闭）
```

✅ 预期结果：version 打印 0.6.3；doctor 如实报告找不到密钥/数据库（本机无微信 4.x，这正是该命令的设计用途——它不是报错，是体检）。

步骤 4 • decoder_lab: 上游真实源码 + 合成图片往返解码 (核心实验)

操作: 进入 `exercise\decoder_lab\` (自建的独立小 crate, `src\decoder\` 三文件为上游源码原样拷贝), `cargo run --release`

目的: 在隐私红线内验证 wx-cli 图片解码管线的正确性——自己造一张 PNG, 自己按微信的格式"加密", 再用上游代码解回来:

- **实验 1:** 合成 87 字节 PNG → 单字节 XOR (key=0x5A) 造伪 legacy .dat → 上游 `dispatch()` 自动反推 key、无损还原 ✓
- **实验 2:** 按上游 v2.rs 注释的真实容器格式 (6B magic + 4B aes_size + 4B xor_size + 1B padding + AES-128-ECB 密文) 手工封 V1 文件, 用固定 key `cfcd208495d565ef` 加密 → 上游解出 decoder=v1_aes、format=png、逐字节一致 ✓
- **实验 3 (负向护栏):** 全零垃圾 → 拒收"无法识别图片 magic"; 空文件 → 拒收"空 .dat 文件"; V2 magic 但缺 image key → 拒收 ✓

```
=== 实验 1: legacy_xor 往返 (合成 PNG → XOR 伪 .dat → 上游 dispatch 解码) ===
```

```
[legacy_xor] decoder=legacy_xor format=png 解码 87 字节
```

```
[legacy_xor] 逐字节比对原图: 一致 ✓
```

```
=== 实验 2: v1_aes 往返 (真实 V1 容器格式 + 固定 key cfcd208495d565ef) ===
```

```
[v1_aes] decoder=v1_aes format=png 解码 87 字节
```

```
[v1_aes] 逐字节比对原图: 一致 ✓
```

```
=== 实验 3: 负向测试 (必须全部被拒) ===
```

```
[全零垃圾] 正确拒绝: legacy XOR: 无法识别图片 magic (key 探测失败) ✓
```

```
[空文件] 正确拒绝: 空 .dat 文件 ✓
```

```
[V2 超长声明] 正确拒绝: V2 .dat: 需要 image AES key (codex 的 image_key 模块尚未填充)
```

```
全部实验通过: 上游 decoder 源码 + 合成数据, 双向解码与负向护栏均符合预期。
```

✓ 预期结果: 三行"一致 ✓" + 三行"正确拒绝 ✓", 收尾"全部实验通过"。

步骤 5 · 读 daemon 缓存目录 (概念验证)

操作: `wx daemon status` (或查看 `~/wx-cli/` 目录结构) 目的: 理解"配置 / 密钥 / socket / 缓存"四件套布局

```
~/wx-cli/
├─ config.json      # 配置
├─ all_keys.json    # 数据库密钥 (本机为空—从未连过真实微信)
├─ daemon.sock / daemon.pid / .log
└─ cache/           # 解密库 + _mtimes.json (mtime 索引)
```

✅ 预期结果: 首次查询自动拉起 daemon 并生成上述目录; 本机因无微信数据, doctor 止步于密钥检查, 属预期行为。

实验结论与边界

- **已实证:** 源码可编译 (MSVC 路线)、二进制可运行、上游图片解码器对合成数据的往返与负向护栏全部正确、daemon 架构从源码读通。
- **未复验 (诚实记录):** macOS SIP/LLDB 取钥、Windows 扫 WeChat.exe 取钥、真实 SQLCipher 库在线打开——三者都需要真实微信环境, 本机不具备, 只做了源码级研读。
- **隐私红线:** 全程零真实微信数据, 与 wechat-ai-memory、yichen-skills 两批次实验同规。